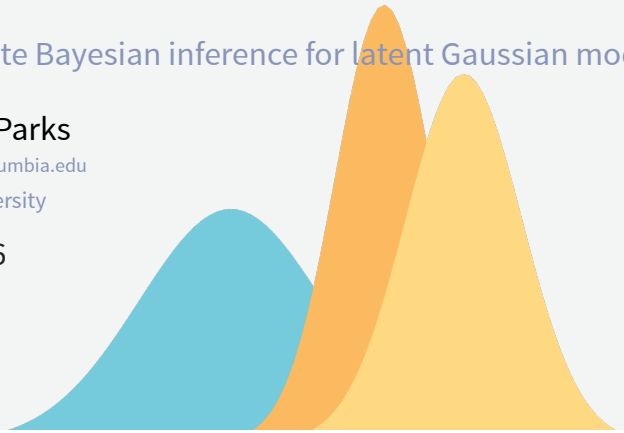
INLA and R-INLA

Approximate Bayesian inference for latent Gaussian models

Robbie M. Parks
robbie.parks@columbia.edu
Columbia University

Aug 6, 2026



Outline

- Recap: where Bayesian inference gets expensive
- What INLA is trading away, and what it buys
- Latent Gaussian models
- Gaussian Markov random fields and sparsity
- The Laplace approximation
- The three steps of the INLA algorithm
- Accuracy, speed, and limits
- `R-INLA`: syntax, `f()`, priors, output
- Pooling models revisited in `INLA`
- Model comparison and spatial extensions

Recap: Bayesian inference

- We have data y, parameters x, and Bayes' theorem:

$$p(x \mid y) = \frac{p(y \mid x)\, p(x)}{p(y)}$$

- The numerator is easy: it is just the likelihood times the prior.
- The denominator $p(y) = \int p(y \mid x) p(x)\, dx$ is the problem.
- For anything beyond conjugate toy examples, that integral has no closed form.

Non-closed solutions

- In a hierarchical model, x is not one parameter but thousands: intercepts, coefficients, one random effect per area, per year, per city-year…
- The normalising constant is then a *very* high-dimensional integral.

Non-closed solutions

- And we rarely want the joint posterior anyway — we want **marginals**:

$$p(x_i \mid y) = \int p(x \mid y)\, dx_{-i}$$

- Which requires integrating out everything else. Also intractable.

The usual answer: MCMC

- Do not compute the integral. **Sample** from the posterior instead.
- Gibbs, Metropolis-Hastings, HMC/NUTS (`NIMBLE`, `Stan`, `JAGS`).
- Guaranteed to be correct *in the limit* of infinite samples.
- Extremely general: works for essentially any model you can write down.
- This is what we did last session with `NIMBLE`.

What MCMC costs

- **Time.** A spatiotemporal model over 8,000 areas and 20 years has $>$ 160,000 latent parameters to sample.
- **Convergence diagnostics.** Trace plots, $\hat{R}$, effective sample size, burn-in, thinning — all need testing and re-testing.

What MCMC costs

- **Autocorrelation.** Strongly correlated latent fields mix badly; effective sample size can be a tiny fraction of the nominal one.
- **Iteration cost.** Every time you change a prior or add a covariate, you wait again.
- Model *development* is where the pain really is: you may fit a model thirty times before you fit it once for real.

Where MCMC starts to hurt

- Disease mapping: one random effect per small area, spatially correlated by construction.
- Space-time interactions: $N_{\text{area}} \times T$ parameters, often weakly identified.
- Cross-validation: refitting the model n times is simply not feasible.
- Sensitivity analysis over priors: same problem again.
- These are exactly the models that climate-and-health work keeps reaching for.

INLA

- **I**ntegrated **N**ested **L**aplace **A**pproximation (Rue, Martino & Chopin, 2009).
- We usually only want **posterior marginals**, not the joint posterior.
- A huge share of the models we actually fit belong to one restricted class — **latent Gaussian models**.
- Within that class, marginals can be *approximated deterministically*, accurately, and very fast.
- No chains. No burn-in. No $\hat{R}$. Same answer every time you run it.

MCMC and INLA

	MCMC	INLA
Approach	Stochastic sampling	Deterministic approximation
Model class	Almost anything	Latent Gaussian models only
Error	Monte Carlo error, $\to 0$ with time	Approximation error, fixed
Runtime	Good! But can be challenge to scale	Potentially accelerated
Reproducible	Up to the seed	Exactly
Diagnostics needed	Many	Few

Latent Gaussian models

The three-stage hierarchy

Every model INLA can fit has this shape:

$$\begin{aligned}
&\textbf{Stage 1:} && y_i \mid x, \theta \sim \prod_{i=1}^{n} p(y_i \mid x_i, \theta) \\
&\textbf{Stage 2:} && \boldsymbol{x} \mid \boldsymbol{\theta} \sim N\left(\mathbf{0}, \boldsymbol{Q}(\boldsymbol{\theta})^{-1}\right) \\
&\textbf{Stage 3:} && \boldsymbol{\theta} \sim p(\boldsymbol{\theta})
\end{aligned}$$

- Stage 1: the likelihood — observations conditionally independent given the latent field.
- Stage 2: the **latent field** x, jointly Gaussian.
- Stage 3: a **small** number of hyperparameters θ.

Stage 1: the likelihood

- Given the latent field, observations are independent:

$$p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{\theta}) = \prod_{i=1}^{n} p(y_i \mid \eta_i, \boldsymbol{\theta})$$

- Each observation depends on the latent field through **one** element: its linear predictor η_i.
- The likelihood itself need **not** be Gaussian: Poisson, binomial, negative binomial, gamma, Weibull, zero-inflated, survival families, etc.
- This is the same conditional independence assumption we always make in hierarchical models.

Stage 2: the latent Gaussian field

- The latent field collects **everything** in the linear predictor:

$$\eta_i = \alpha + \sum_k \beta_k z_{ki} + \sum_j f_j(u_{ji})$$

- $\boldsymbol{x} = \{\boldsymbol{\eta}, \alpha, \boldsymbol{\beta}, \boldsymbol{f}\}$
- Intercept, fixed effects, random effects, smooth functions, spatial fields, temporal trends — all of it.
- All assigned a **joint Gaussian prior** with precision matrix $Q(\theta)$.
- Note: fixed effects get vague Gaussian priors, so they are Gaussian too. Nothing is excluded.

Stage 3: the hyperparameters

- θ contains everything **not** Gaussian:
  - precisions (inverse variances) of random effects,
  - correlation and range parameters,
  - overdispersion, mixing weights, likelihood-specific parameters.
- These enter $Q(\theta)$ and the likelihood.
- Crucially, $\dim(\theta)$ must be **small** — typically 1 to 6, rarely more than 10.
- The latent field can be enormous; the hyperparameter space cannot be.

What counts as a latent Gaussian model?

Almost everything we fit in this course:

- Linear and generalised linear models
- Mixed / multilevel models (IID random effects)
- Temporal models: RW1, RW2, AR(p), seasonal
- Smoothing splines and non-linear exposure-response (`rw2`, `crossbasis`-style)
- Spatial models: ICAR, BYM, BYM2, Leroux, SPDE/Matérn
- Spatiotemporal models, including interaction types I-IV
- Measurement error, missing covariates, survival models

Two conditions must hold: the latent field is **Gaussian and sparse**, and θ is **low-dimensional**.

Gaussian Markov random fields

- A GMRF is a Gaussian vector whose **precision matrix** $Q = \Sigma^{-1}$ is sparse.
- The Markov property:

$$Q_{ij} = 0 \iff x_i \perp x_j \mid \boldsymbol{x}_{-ij}$$

- Conditional independence $\Rightarrow$ zeros in Q.
- Neighbours-only spatial models (ICAR) and random walks are GMRFs by construction.
- Sparsity is important: factorising a sparse Q is $O(n^{3/2})$ in 2D rather than $O(n^3)$, so a 10,000-area field costs a fraction of a second, not hours.

How INLA works (briefly)

What we want

Two sets of marginals:

$$p(x_i \mid \boldsymbol{y}) = \int p(x_i \mid \boldsymbol{\theta}, \boldsymbol{y})\, p(\boldsymbol{\theta} \mid \boldsymbol{y})\, d\boldsymbol{\theta}$$

$$p(\theta_j \mid \boldsymbol{y}) = \int p(\boldsymbol{\theta} \mid \boldsymbol{y})\, d\boldsymbol{\theta}_{-j}$$

- Note the **nesting**: the latent marginal is an average of conditional marginals, weighted by the hyperparameter posterior.
- So we need three things: $p(\boldsymbol{\theta} \mid \boldsymbol{y})$, $p(x_i \mid \boldsymbol{\theta}, \boldsymbol{y})$, and a way to integrate.
- That is the “integrated nested” part of the name.

The Laplace approximation

- Old idea (Laplace, 1774): approximate an integral by fitting a Gaussian at the mode of the integrand.
- Take logs, expand to second order around the mode x^*:

$$\log f(\boldsymbol{x}) \approx \log f(\boldsymbol{x}^*) - \tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\top \boldsymbol{H} (\boldsymbol{x} - \boldsymbol{x}^*)$$

- The result is Gaussian, so the integral is available in closed form.
- It works well when the target is unimodal and reasonably symmetric — which posteriors of latent Gaussian fields usually are.

Laplace approximation, formally

$$\int f(\boldsymbol{x})\, d\boldsymbol{x} \approx f(\boldsymbol{x}^*)\, (2\pi)^{d/2}\, |\boldsymbol{H}|^{-1/2}$$

- x^*: the mode, found by Newton-Raphson.
- H: the negative Hessian at the mode.
- The Gaussian prior on x helps enormously: the target is already "nearly Gaussian" before we start.

The three steps, in words

Think of it as: **if I knew the variances, the rest would be easy.**

1. **Get a posterior for the few hyperparameters** θ (the variances, correlations).
2. **Pick a handful of plausible values of** θ to stand in for the whole distribution.
3. **Fit the latent field at each one**, then average the answers, weighting each by how plausible that θ was.

- Step 3 is cheap because, with θ held fixed, the latent field is nearly Gaussian.
- Steps 1-2 are cheap because θ has only a few entries.

Step 1: a posterior for the hyperparameters

We want $p(\theta \mid y)$: how plausible is each set of variances, given the data?

- We can write down the **joint** $p(x, \theta, y)$ exactly — it is just likelihood $\times$ priors.
- Divide the joint by $p(x \mid \theta, y)$ and the latent field cancels out, leaving what we want.
- Problem: that denominator is the thing we cannot compute.
- Solution: **swap it for a Gaussian approximation** — which is a reasonable stand-in, because a latent Gaussian field with θ fixed really is close to Gaussian.

Step 1: the same thing in symbols

$$\tilde{p}(\theta \mid y) \propto \left.\frac{\overbrace{p(y \mid x, \theta)\, p(x \mid \theta)\, p(\theta)}^{\text{joint: we know this exactly}}}{\underbrace{\tilde{p}_G(x \mid \theta, y)}_{\text{Gaussian stand-in}}}\right|_{x = x^*(\theta)}$$

- The identity holds for **any** value of x, so we are free to choose one — and we choose the mode $x^*(\theta)$, where the Gaussian stand-in is most accurate.
- This is exactly the Laplace approximation from two slides ago, applied to the latent field.
- Output: a function we can **evaluate** at any θ, though we have no formula for it.

Step 2: where to evaluate θ

We can only evaluate $\tilde{p}(\theta \mid y)$ point by point, so we need to choose good points.

- Climb to the **top** of the surface (Newton-Raphson).
- Measure the **curvature** at the top (the Hessian) — this tells us how wide the distribution is in each direction.
- Use that to rescale the axes, so the surface is roughly round rather than a long thin ridge.
- Then lay down evaluation points on the rescaled, well-behaved surface.

Step 2: how many points?

`int.strategy`	What it does	Trade-off
`"grid"`	Dense grid over θ	Most accurate; cost explodes beyond 2-3 hyperparameters
`"ccd"`	A clever scatter of points around the mode	**Default** when $\dim(\theta) > 2$; good accuracy, few points
`"eb"`	The mode only	Fastest; ignores uncertainty in θ

- Each chosen point $\theta^{(k)}$ comes with a weight: how plausible it is.
- Useful during model development: run `"eb"` while you iterate, then `"ccd"` for the final fit.

Step 3: averaging over the hyperparameters

For each $\theta^{(k)}$ we fit the latent field, then take a **weighted average** of the results:

$$\underbrace{\tilde{p}(x_i \mid y)}_{\text{what we report}} \approx \sum_k \underbrace{\tilde{p}(x_i \mid \theta^{(k)}, y)}_{\text{answer if } \theta = \theta^{(k)}} \times \underbrace{\tilde{p}(\theta^{(k)} \mid y)\, \Delta_k}_{\text{how much it counts}}$$

- Same spirit as multiple imputation: solve the easy problem several times, then average, so uncertainty in θ is carried through.
- The result is a **density**, not a set of samples.
- One thing still missing: how to get each $\tilde{p}(x_i \mid \theta^{(k)}, y)$.

Three strategies for the latent marginals

- **Gaussian** (`strategy = "gaussian"`): read the marginal straight off the Gaussian approximation. Fastest; can be poor in the tails and miss skewness.
- **Laplace** (`strategy = "laplace"`): a full nested Laplace approximation for each x_i. Most accurate, most expensive.
- **Simplified Laplace** (`strategy = "simplified.laplace"`): a third-order Taylor correction to the Gaussian, capturing location and skewness. The default, and almost always the right choice.

The algorithm in one slide

1. Find the mode of $\tilde{p}(\boldsymbol{\theta} \mid \boldsymbol{y})$ via Newton-Raphson.
2. Compute the Hessian; build the standardised z-space.
3. Place integration points (grid or CCD) and evaluate $\tilde{p}(\boldsymbol{\theta}^{(k)} \mid \boldsymbol{y})$.
4. At each point, compute $\tilde{p}(x_i \mid \boldsymbol{\theta}^{(k)}, \boldsymbol{y})$ for all i.
5. Sum over k with weights to get $\tilde{p}(x_i \mid \boldsymbol{y})$.
6. Marginalise $\tilde{p}(\boldsymbol{\theta} \mid \boldsymbol{y})$ for each θ_j.

Why it is fast

- No sampling, so no autocorrelation and no wasted iterations.
- Sparse matrix algebra on $Q(\boldsymbol{\theta})$: Cholesky factorisation exploiting the Markov structure.
- The expensive work scales with $\dim(\boldsymbol{\theta})$, which is small — **not** with the size of the latent field, which is large.
- Everything at a given $\boldsymbol{\theta}^{(k)}$ is parallel (`num.threads`).

How accurate is it?

- For latent Gaussian models with well-behaved likelihoods, INLA and long MCMC runs typically agree to two or three decimal places on posterior means and quantiles.
- Accuracy degrades when:
  - the number of observations per latent parameter is very small (e.g. binary data with one observation per random effect),
  - the likelihood is far from Gaussian and weakly informative,
  - the posterior is genuinely multimodal.
- Check with `inla(..., control.compute = list(config = TRUE))` and compare against a short MCMC run when it matters.

When not to use INLA

- The latent field is **not** Gaussian (e.g. discrete mixture components, changepoint indicators).
- $\dim(\theta)$ is large — say more than 10-15.
- You need the **joint** posterior over many latent parameters, not marginals. (Partly recoverable via `inla.posterior.sample()`.)
- Genuinely multimodal posteriors.
- Novel likelihoods with no INLA implementation — though `rgeneric` and `cgeneric` let you write your own.

Using R-INLA

Installation

- INLA is not on CRAN. Install from the project repository:

```r
install.packages(
  "INLA",
  repos = c(getOption("repos"),
            INLA = "https://inla.r-inla-download.org/R/stable"),
  dep = TRUE
)

library(INLA)
inla.version()
```

- Use `testing` instead of `stable` for the development version.
- `inla.upgrade()` to update in place.

The central function: `inla()`

```r
formula <- y ~ 1 + x1 + x2 + f(area, model = "iid")

m <- inla(
  formula,
  family            = "poisson",
  data              = data,
  E                 = data$expected,
  control.compute   = list(dic = TRUE, waic = TRUE, cpo = TRUE),
  control.predictor = list(compute = TRUE, link = 1)
)

summary(m)
```

What the formula means

- `y ~ ...` on the scale of the linear predictor.
- Everything on the right-hand side that is **not** wrapped in `f()` is a fixed effect (Gaussian prior, vague by default).
- Everything wrapped in `f()` is a structured latent component.
- Offsets: use `E =` for Poisson (expected counts) or `Ntrials =` for binomial. Note `E` is on the **natural** scale, not logged.
- `−1` removes the intercept, exactly as in `lm()`.

The `f()` function

```
f(index,
  model      = "iid",       # latent model class
  hyper      = list(prec = list(prior = "pc.prec",
                                param = c(1, 0.01))),
  constr     = TRUE,        # sum-to-zero constraint
  graph      = "map.adj",   # for spatial models
  replicate  = id,          # independent copies
  group      = time)        # grouped/interaction structure
```

- `index` **must** be an integer or factor indexing the latent component — not the covariate value itself.
- Each `f()` term adds hyperparameters to θ.

Some latent models available

`model =`	Structure	Typical use
`"iid"`	Independent Gaussian	Unstructured group effects
`"rw1"`, `"rw2"`	Random walk	Smooth time trends, exposure-response
`"ar1"`	Autoregressive	Serially correlated time series
`"besag"`	ICAR	Spatial, neighbours only
`"bym2"`	Reparameterised BYM	Disease mapping (recommended)
`"spde2"`	Matérn via SPDE	Point-referenced / continuous space
`"generic0"`-`"generic3"`	User-supplied Q	Custom structures

`inla.list.models()` gives the full list.

Complete pooling in INLA

Recall from the lecture:

$$y_i \sim \text{Pois}(\mu_i) \quad i = 1, \ldots, N$$
$$\log(\mu_i) = \log(E_i) + \alpha$$

```
m_pooled <- inla(
  deaths ~ 1,
  family = "poisson",
  E      = expected,
  data   = data,
  control.compute = list(dic = TRUE, waic = TRUE)
)
```

- One intercept, shared by every province. No `f()` term, so θ is empty.

No pooling in INLA

$$y_i \sim \text{Pois}(\mu_i)$$
$$\log(\mu_i) = \log(E_i) + \alpha_{j[i]}$$

```
m_unpooled <- inla(
  deaths ~ -1 + factor(province),
  family = "poisson",
  E      = expected,
  data   = data,
  control.fixed = list(prec = 1),   # N(0, 1) on each alpha_j
  control.compute = list(dic = TRUE, waic = TRUE)
)
```

- Province enters as a **fixed effect**: independent parameters, no borrowing of strength.
- `-1` drops the global intercept so each level is estimated directly.

Partial pooling in INLA

$$\log(\mu_i) = \log(E_i) + \alpha + \theta_{j[i]}, \quad \theta_j \sim N(0, \sigma_p^2)$$

```
data$province_id <- as.integer(as.factor(data$province))

m_partial <- inla(
  deaths ~ 1 + f(province_id, model = "iid",
                 hyper = list(prec = list(
                   prior = "pc.prec", param = c(1, 0.01)))),
  family = "poisson",
  E      = expected,
  data   = data,
  control.compute = list(dic = TRUE, waic = TRUE, cpo = TRUE)
)
```

- One line of difference from the pooled model.

Reading the output: fixed effects

```
m_partial$summary.fixed
```

- Columns: `mean`, `sd`, `0.025quant`, `0.5quant`, `0.975quant`, `mode`, `kld`.
- `kld` is the Kullback-Leibler divergence between the Gaussian and the simplified Laplace marginal. Values much above ~ 0.01 suggest the approximation is straining.
- Exponentiate for rate ratios — but do it on the **marginal**, not on the summary, if you want a correct interval.

Reading the output: hyperparameters

```
m_partial$summary.hyperpar
```

- `INLA` reports **precisions**, not standard deviations: $\tau = 1/\sigma^2$.
- A posterior mean of a precision is *not* the reciprocal of the posterior mean of a variance. Transform the whole marginal:

```
sd_marg <- inla.tmarginal(function(x) 1 / sqrt(x),
                          m_partial$marginals.hyperpar$`Precision for provi
inla.zmarginal(sd_marg)
inla.emarginal(function(x) x, sd_marg)   # posterior mean of sigma
inla.hpdmarginal(0.95, sd_marg)          # HPD interval
```

Random effects and shrinkage

```
re <- m_partial$summary.random$province_id

ggplot(re, aes(x = ID, y = mean)) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_pointrange(aes(ymin = `0.025quant`, ymax = `0.975quant`)) +
  labs(x = "Province", y = expression(theta[j]))
```

- Compare the spread of these to the unpooled `factor(province)` estimates: the hierarchical ones are pulled toward zero.
- Provinces with small expected counts shrink most — adaptive regularisation, estimated from the data.

Priors: what INLA does by default

- Intercept: $N(0, 1000^2)$ — effectively flat.
- Other fixed effects: $N(0, 1000^2)$ by default; set via `control.fixed = list(mean = 0, prec = 0.01)`.
- Random effect precisions: `loggamma(1, 5e-5)` on the log-precision.
- Always state your priors in the paper. Always run a sensitivity analysis.

PC priors

Penalised complexity priors (Simpson et al., 2017) shrink toward a simpler *base model* — here, $\sigma = 0$ — with an exponential penalty on the distance from it.

```
hyper = list(prec = list(prior = "pc.prec", param = c(U, alpha)))
```

- Read as $P(\sigma > U) = \alpha$.
- `param = c(1, 0.01)`: "we think it is unlikely (1% chance) that the between-group SD exceeds 1 on the log scale".
- Interpretable, weakly informative, and conservative — they do not invent structure that is not in the data.
- Recommended default for all variance components.

Model comparison: DIC, WAIC, CPO

```
map_dbl(list(m_pooled, m_unpooled, m_partial), ~ .x$dic$dic)
map_dbl(list(m_pooled, m_unpooled, m_partial), ~ .x$waic$waic)

# Leave-one-out log score (lower is better)
-mean(log(m_partial$cpo$cpo), na.rm = TRUE)

# Check the CPO computation is trustworthy
sum(m_partial$cpo$failure > 0)
```

- `dic$p.eff` and `waic$p.eff`: effective number of parameters, a nice numerical read on how much shrinkage is happening.
- PIT values (`m$cpo$pit`) should be roughly uniform if the model is calibrated.

Two-level IID model in INLA

City effects and year-within-city effects, as in the lecture:

$$\log(\mu_{jt}) = \log(E_{jt}) + \alpha + \beta x_{jt} + \theta_j + \gamma_{jt}$$

```r
d$city_id     <- as.integer(as.factor(d$city))
d$cityyear_id <- as.integer(as.factor(paste(d$city, d$year)))

pc <- list(prec = list(prior = "pc.prec", param = c(1, 0.01)))

m2 <- inla(
  admissions ~ 1 + pm25 +
    f(city_id,     model = "iid", hyper = pc) +
    f(cityyear_id, model = "iid", hyper = pc),
  family = "poisson", E = expected, data = d,
  control.compute = list(dic = TRUE, waic = TRUE)
)
```

- Two hyperparameters. Two `f()` terms.

Spatial extension: BYM2

```r
nb <- poly2nb(shp)
nb2INLA("map.adj", nb)
g <- inla.read.graph("map.adj")

m_bym2 <- inla(
  deaths ~ 1 + f(area_id, model = "bym2", graph = g,
                 scale.model = TRUE,
                 hyper = list(
                   prec = list(prior = "pc.prec", param = c(1, 0.01)),
                   phi  = list(prior = "pc",      param = c(0.5, 0.5)))),
  family = "poisson", E = expected, data = shp,
  control.compute = list(dic = TRUE, waic = TRUE)
)
```

- `bym2` separates total SD from the **mixing parameter** ϕ: the proportion of variance that is spatially structured.
- `scale.model = TRUE` makes the precision comparable across maps — essential for the prior to mean anything.

Practical tips and common errors

- *"Index for f() must be integer or factor"* — you passed a covariate value. Create an explicit `1:n` index.
- Missing values in `y` are **predicted**, not dropped. Useful, but be deliberate about it.
- Model does not converge: try `control.inla = list(strategy = "gaussian", int.strategy = "eb")` first to check the model runs, then tighten.
- `control.inla = list(h = 0.01)` if the numerical Hessian misbehaves.
- Centre and scale continuous covariates: it helps the optimiser and makes priors interpretable.
- `verbose = TRUE` prints what the optimiser is actually doing.

INLA vs NIMBLE

	NIMBLE	INLA
Inference	MCMC (exact in the limit)	Deterministic approximation
Model class	Anything you can write in BUGS	Latent Gaussian models
Spatiotemporal model	Hours	Seconds to minutes
Priors	Fully flexible	Gaussian latent field required
Diagnostics	$\hat{R}$, ESS, traces	`kld`, CPO failures, PIT
Best for	Bespoke, non-standard models	Standard hierarchical/spatial models, model development

- We will see models in `INLA` going forward

Getting ready for the lab

The lab for this session

- This lab revisits the Italian mortality data from the hierarchical modelling session, using `R-INLA` instead of `NIMBLE`.
- During this lab session, we will:

1. Refit the pooled, unpooled and partially pooled models in `INLA`;
2. Compare estimates and runtimes against the `NIMBLE` output;
3. Explore priors on the variance components; and
4. Extend the model to a spatially structured `bym2`.

Questions?